チーム開発のコードの書き方

登壇者:alion

まず最初に

今はAIが発展しすぎていて、この話は無駄かもしれない。

でも、知っているか知らないかでAIへの指示の出し方も変わる。

これは私の思う「チームのためのコード」です。
皆さんも自分なりの「チームのためのコード」を定義してほしい。

自己紹介

HN: αlion, LandEater(旧), ふくらP

趣味: ガンダム TCG メカ ロボット

トレカは現在5タイトルくらいやってます



役職:プログラマ

最近のトピック

#白衣 #ガンダム #Blenderできるようになりたい #時間ない
#鍋は神 #時間ない #抹茶 #梅干し #Dr. pepper #バトオペ3
#時間ない #初音ミクのmetalbuild欲しい #AC6やる時間欲しい
#メカ好き #逆関節って叡智だと思う #理論よりロマン #時間ない
ゴロリ

コンテンツ

- チーム開発とは
- チームで定められたコード規則を守る
- 命名をきちんとする
- 自分の知らないコードを読む。

チーム開発って
なんですか？

チーム開発とは

- 他の人と一緒にコードを書く。

- 自分のコードが他人に読まれる。

- 自分の知らないコードを読む

チーム開発で大切なこと

- チームで定められたコード規則を守る

- 命名をきちんとする

- 処理を分ける

チームで定められた規則を守る

チームで定められたコード規則を守る

フィールド：_camelCase / 原則 private

```
private int _currentHp;
private GameObject _playerPrefab;
[SerializeField] private float _moveSpeed = 10f;
```

引数・ローカル変数：camelCase　／　定数：PascalCase

```
void TakeDamage(int damageAmount)
{
    int remainingHp;
}
private const int MaxEnemyCount = 100;
public const string PlayerTag = "Player";
```

外部への公開はプロパティやメソッドで。 例：public int CurrentHp => _currentHp;

```
public class Player
{
    public float Hp = 100f;

    public void TakeDamage(float damage)
    {
        Hp = Mathf.Max(0f, Hp - damage);
        // ダメージ演出やシステムへの通知も行うとしたら？
    }
}
```

外から Hp = -999f; と変更できてしまう。→ バグの原因に！

```
public class Player
{
    private float _hp = 100f;
    public float Hp => _hp;

    public void TakeDamage(float damage)
    {
        _hp = Mathf.Max(0f, _hp - damage);
        // ダメージ演出や通知
    }
}
```

別に意地悪で書いているわけではないので守ってほしい...

命名をきちんとする

命名をきちんとする

悪い例：名前から何をする処理か分からない

```
public void Apply(int value)
{
    _hp = value;
}
```

これ、最大HP？ 現在HP？
ダメージ処理？ 回復処理？
何を適用しているの？

良い例：処理の目的が名前から分かる

例えば、名前で意図が分かるようにする。

```
public void SetMaxHp(int value)
{
    _maxHp = value;
}
```

```
class StudentData
{
    private string _name;
    private int _year;
    private string _studentId;
}
class DataBase { void Show() { /* ... */ } }
```

ちなみに、私の本名は堀口陸です。
あとHNはαlionとLandEaterとふくらPです。

```
class StudentData
{
    private string _name;
    private int _year;
    private string _studentId;
}
class DataBase { void Show() { /* ... */ } }
```

name は苗字？ 名前？ HN？
year は学年？ 年齢？
DataBase は何のデータベース？ Show は何を表示？

```
class StudentData
{
    private string _fullName;
    private int _schoolYear;
    private string _studentId;
}
class StudentList { void ShowAll() { /* ... */ } }
```

命名を定めるだけで、読む人の誤解を減らせる。

処理を分ける

処理を分ける

```
public void TakeDamage(int damage)
{
    _currentHp -= damage;
    if (_currentHp < 0) _currentHp = 0;
    _hpBar.value = (float)_currentHp / _maxHp;
    if (_currentHp == 0)
    {
        Debug.Log("死亡！");
        Destroy(gameObject);
    }
}
```

死亡演出を追加したいとき、どこを修正する？

```
public void TakeDamage(int damage)
{
    _currentHp = Mathf.Max(0, _currentHp - damage);
    UpdateHpBar();
    CheckDeath();
}
private void UpdateHpBar() { _hpBar.value = (float)_currentHp / _maxHp; }
private void CheckDeath()
{
    if (_currentHp > 0) return;
    Debug.Log("死亡！");
    Destroy(gameObject);
}
```

関数名だけで、処理の流れが分かる。

```
private void SpawnEnemy()
{
    if (_currentEnemyCount >= _maxEnemyCount)
        return;
    if (_enemyPrefabs.Length == 0) return;
    int index = Random.Range(
        0, _enemyPrefabs.Length);
    GameObject prefab = _enemyPrefabs[index];
    float x = Random.Range(-20f, 20f);
    float z = Random.Range(-20f, 20f);
    Vector3 position = new Vector3(x, 0f, z);
    if (Vector3.Distance(
        position, _player.position) < 5f)
        position += Vector3.forward * 10f;
    GameObject enemy = Instantiate(
        prefab, position, Quaternion.identity);
```

```
    Enemy component =
        enemy.GetComponent<Enemy>();
    if (component != null)
    {
        int hp = 100;
        int attack = 10;
        if (_currentWave >= 5)
        { hp += 50; attack += 5; }
        if (_currentWave >= 10)
        { hp += 100; attack += 10; }
        component.SetStats(hp, attack);
    }
    if (_spawnEffect != null)
        Instantiate(_spawnEffect,
            position, Quaternion.identity);
    if (_audioSource != null &&
        _spawnSound != null)
        _audioSource.PlayOneShot(_spawnSound);
    _currentEnemyCount++;
    Debug.Log("敵をスポーンしました");
}
```

処理を分ける

まとめ

- チームで定められたコード規則を守る

- 命名をきちんとする

- 処理を分ける

これを自分の中で噛み砕いて、
自分なりの「チームのためのコード」を定義してほしい。